ScintillaOS

BuildStorm 题二 2.3

内核设计与优化实现文档



学校：武汉大学

队员：雷雨泽、杜宇恒、宋天琦

二〇二六年八月

摘　　要

本文说明 ScintillaOS 为全国大学生操作系统比赛 2026 年线上决赛题二 BuildStorm 所进行的兼容性开发、并发正确性修复和性能优化。目标负载不是单个定制程序，而是在 glibc 用户态中运行真实 Rust 工具链，以多进程、多线程方式从源码构建数百个 crate，最终生成并启动 arceos-helloworld。该工作负载同时覆盖动态链接、进程和线程、futex、epoll、虚拟内存、ext4、VirtIO 块设备、多核调度和双架构页表等内核子系统。

2026 年 8 月 12 日官网最终日志证明：RISC-V64 与 LoongArch64 均通过工具链检查、最小项目构建、完整 release 构建、产物大小检查和产物启动检查。RISC-V64 的正式构建耗时为 2270.92 秒，LoongArch64 为 1700.89 秒；BuildStorm 双平台原始得分合计 311.4 分，前置 CAgent 回归双平台均为 199.1 分，整次官网评测总分为 709.6 分。本文给出从故障定位到最终结果的完整证据链，并明确区分最终保留的实现与实验后回退的候选方案。

关键词：ScintillaOS；操作系统内核；BuildStorm；Rust；RISC-V；LoongArch；性能优化

目　录

1 评审范围与最终结论 **1**

1.1 四项人工评审要求的对应位置 1

1.2 最终官网结果 1

2 问题定位与根因分析 **3**

2.1 工作负载、约束与总体架构 3

2.1.1 工作负载不是面向单一用例的适配 3

2.1.2 最终数据路径 3

2.2 定位过程与根因 4

2.2.1 定位方法：先短测，再长测，最后官网闭环 4

2.2.2 启动速度和时间源的根因 4

2.2.3 多线程进程语义缺口 4

2.2.4 用户内存访问与 COW 竞态 5

2.2.5 文件系统是主要性能瓶颈 5

2.2.6 SMP 锁与内存分配瓶颈 5

2.2.7 LoongArch 架构特有根因 6

2.2.8 编译成功后仍可能丢失全部分数 6

3 修复或优化的设计与实现 **7**

3.1 核心修复与优化设计 7

3.1.1 时间、计时与热中断路径 7

3.1.2 进程、线程、fd 与等待语义 7

3.1.3 futex、epoll 与 IPC 7

3.1.4 虚拟内存、按需装入与 COW 8

3.1.5 VFS 语义与缓存一致性 8

3.1.6 文件页缓存、块缓存与 I/O 合并 9

3.1.7 allocator、调度器与锁 9

3.1.8 LoongArch 页表、TLB 与设备 9

3.1.9 最终产物启动与确定性构建 10

3.2 题 2.2 修改覆盖矩阵 10

4 实验分析 **15**
4.1 实验口径 15
4.2 最终保留优化的逐项 A/B 15
4.3 并发故障的阶段性证据 17
4.4 反向实验与回退决策 17
4.5 完整长测与官网结果 18

5 AI 使用说明与可复现步骤 **20**
5.1 AI 辅助使用说明 20
5.2 可复现开发过程与最终验收 21
5.2.1 复现材料、版本和不变量 21
5.2.2 从基线重演提交演进 21
5.2.3 建立干净输入并采集定位证据 22
5.2.4 按同条件 A/B 重演优化决策 22
5.2.5 重建最终产物并执行官方双盘验收 23
5.3 总结与后续方向 24

1　评审范围与最终结论

1.1　四项人工评审要求的对应位置

评审项	本文提供的证据	主要章节
问题定位与根因分析（6 分）	系统调用级故障、锁等待、块设备计数、页表和 QEMU monitor 证据；区分宿主资源不足与 guest 内核错误	第 2 章
修复或优化的设计与实现（6 分）	Linux 语义下的进程、内存、文件系统、锁、双架构和产物启动设计；完整修改覆盖矩阵	第 3 章
实验分析（4 分）	同条件 A/B、回退实验、完整本地长测和官网最终数据；报告耗时、请求数、锁等待和加速比	第 4 章
AI 使用说明与可复现步骤（4 分）	AI 的真实参与边界、错误修正方式、源码和日志留痕、可重演的开发流程与双架构最终验收	第 5 章

表 1.1　人工评审项索引

1.2　最终官网结果

最新官网日志为 `Riscv -11.txt` 和 `LoongArch -11.txt`。两份日志都出现 `TOOLCHAIN_RESULTstatus=OK`、`MINIBUILD_RESULTstatus=OK` 和最终的 `BUILDSTORM_RESULT...status=OK...run=OK`，随后正常输出测试组结束标记并关机。

平台	核数	正式构建耗时	产物大小	自动评测得分	产物启动
RISC-V64	8	2270.92 s	1,681,000 B	131.4 / 200	OK
LoongArch64	12	1700.89 s	1,714,568 B	180.0 / 200	OK
双平台合计	—	—	—	311.4 / 400	均通过

表 1.2　2026-08-12 官网 BuildStorm 自动评测最终结果

公开 judge 的 Linux 基线为 RISC-V64 的 1616.09 秒和 LoongArch64 的 1985.21 秒。编译时间分按

$$S_{time} = 120 \times \mathrm{clamp}\left(\frac{2B - t}{B}, 0, 1\right)$$

计算。RISC-V64 的 2270.92 秒对应 71.4 分；LoongArch64 的 1700.89 秒快于其基线，得到 120 分。两平台还分别获得工具链 8 分、最小构建 12 分和完整构建 40 分。题 2.3 的 20 分为本文对应的人工评审项，不在上述脚本自动得分中。同一轮 CAgent 十项在两平台均全通过且均获得 199.1 分，说明 BuildStorm 改动没有破坏前置基础命令、网络和文件系统能力。

2 问题定位与根因分析

2.1 工作负载、约束与总体架构

2.1.1 工作负载不是面向单一用例的适配

题二首先运行 `rustc--version`、`cargo--version`，再用 `cargonew` 创建项目、执行真实 `cargobuild` 并运行 Hello World；随后从零构建 arceos-helloworld。完整构建包含 446 个左右的预构建单元和数百个 crate，持续产生 `fork/clone/exe cve/wait4`、futex、epoll、pipe、Unix socket、mmap、COW、文件创建、rename、truncate、目录遍历和同步块 I/O。我们没有识别 crate 名称、文件名或评分脚本来跳过工作，也没有修改 `/proc/uptime` 或系统时钟伪造耗时。

正式环境同时要求：保留 VirtIO 网卡；RISC-V64 和 LoongArch64 使用相同 Linux ABI；文件系统镜像离线且可写；QEMU 为大内存多核配置；最终产物必须超过 500 KiB 并实际启动。因此 “cargo 返回 0” 不是充分条件，必须保证整个产物链路可复现。

2.1.2 最终数据路径

`bash/cargo/rustc/linker/nestedQEMU`
↓ Linux ABI：进程、线程、futex、epoll、socket、mmap、文件系统系统调用
↓ Task / Scheduler / Signal / VM / VFS / Page Cache
↓ lwext4 事务与全局哈希加 CLOCK 块缓存
↓ VirtIO Block；并行保留 VirtIO Net
↓ RISC-V64 Sv39 或 LoongArch64 页表、TLB 与 IPI

内核把正确性边界放在 Linux ABI、地址空间和 ext4 事务层，把性能边界放在文件页缓存、块缓存、高低水位写回、分片 allocator 和按需调度上。这样优化不是对某个测试命令调参，而是降低真实复杂软件构建的通用成本。

2.2 定位过程与根因

2.2.1 定位方法：先短测，再长测，最后官网闭环

早期单次完整构建需要数小时，直接长测会把定位周期拉到不可接受。我们建立了四层证据：第一层是 `cargocheck`、双架构内核构建和 ABI 静态审计；第二层是 syscall 语义小测、`FS_QUICK`、COW、VM、SMP、公平性和嵌套 QEMU 启动测试；第三层是真实官方磁盘的工具链、minibuild、446 单元预构建和 release 构建；第四层才是官网 judge 输出。

诊断构建记录页表锁、帧分配器、Rust heap、调度器、VFS 和块设备的调用、竞争、累计等待、最长等待、DMA 字节数与请求大小。长时间停顿时通过 QEMU monitor 的 CPU 快照判断所有 hart 是在用户态计算、设备等待、锁等待还是 guest panic。宿主 Docker/Rosetta 的资源终止不被当作内核语义失败；只有新鲜覆盖层上可重复的 guest 证据才进入修复结论。

2.2.2 启动速度和时间源的根因

最初内核把实时时钟偏移混入运行时间，`/proc/uptime` 也缺少稳定实现。BuildStorm 以 guest uptime 计时，工具链内部的超时又依赖单调时钟；两者混用会使墙钟校准影响超时和评分。另一个大头是启动阶段直接触碰或清零大块物理内存，并在 `execve` 时立即读取全部 ELF 段和分配完整用户栈。Rust 工具链频繁启动短命令，这些工作大部分从未被访问。

2.2.3 多线程进程语义缺口

真实 glibc 和 Rust runtime 会组合使用 `CLONE_VM`、`CLONE_FILES`、`CLONE_THREAD`、`clear_child_tid`、futex requeue、`FD_CLOEXEC` 和 wait。旧实现把“线程 ID”“进程 ID”“子进程所有者”和“地址空间所有者”混在一起，导致：同一线程组中的非 leader 无法等待组内创建的子进程；`execve` 后共享 fd 表生命周期错误；退出线程没有正确清零并唤醒 `clear_child_tid`；信号唤醒与 Ready–Running 交接之间还可能把正在运行的任务重复入队。

futex 的旧 key 只使用用户虚拟地址。不同进程恰好使用相同虚拟地址时会错误互相唤醒；反过来，同一 `MAP_SHARED` 页在不同地址映射时又不能共享等待队

列。requeue 或超时后，任务还可能残留在旧 key 中，最终使 cargo 永久等待。首次完整 RV 构建在 `0/446` 出现 `pid...wasenqueuedtwice`，正是信号提升优先级时未重新核对任务状态的竞态。

2.2.4 用户内存访问与 COW 竞态

旧的 syscall 写回路径先检查用户页可写，再释放 VM 锁执行实际复制。另一个线程可在两步之间 `fork`，把同一 PTE 改成只读 COW；内核随后写入旧翻译结果会得到 `EFAULT`，表现为 cargo 指纹文件、`execve` 参数或 socket 地址随机出现 `Badaddress`。这不是简单增加一次重试可以完全解决的问题，因为校验、触发 COW 和复制必须属于同一个地址空间同步协议。

`MADV_DONTNEED` 原先只改变 VMA 元数据，没有移除 PTE 和释放物理页；8 GiB 环境下 rustc 反复映射和丢弃内存，最终形成确定性的常驻页增长。共享 VM 的多核任务还要求在 `mprotect`、`munmap`、COW 和 `mremap` 后向所有正在运行该地址空间的 CPU 发送 TLB shootdown，否则远端 hart 可继续使用旧写权限或已释放物理页。

2.2.5 文件系统是主要性能瓶颈

初始 ext4 路径的三个大头是：每个 4 KiB miss 都线性扫描固定块缓存；脏块逐块同步写回；每次 `stat/open/lookup` 都重复走完整路径并进入 lwext4 全卷锁。Rust 构建包含大量小文件、元数据查询、临时文件 rename 和顺序读取，导致 CPU 数增加后反而在同一把卷锁和 VirtIO 队列前排队。

语义上也存在会中断构建的问题。目录 rename 的链接计数按“新增目录”处理会破坏父目录计数；`ftruncate` 后的 `stat` 错把当前文件偏移当作文件大小；同一 inode 的多个打开实例没有共享最新 size；稀疏写、打开后 unlink、rename 替换和目录收缩后的迭代器 EOF 边界不完整。只优化缓存而不修这些语义，会更快地产生错误产物。

2.2.6 SMP 锁与内存分配瓶颈

内核早期从单核 `UPSafeCell` 迁移到 SMP 锁后，仍存在两个结构性问题。一是所有动态元数据都由单一 Rust buddy heap 管理，四路 rustc 下该锁累计等待超

过 13 秒；大量进程同时退出时集中释放页表和 VMA，1 GiB 压力段需要 30.98 秒。二是睡眠读写锁的等待条件复查调用 `try_read/try_write`，它会临时获取后释放锁并改变唤醒序列，却没有对应通知，LoongArch 预构建稳定停在 `438--445/446`。

仅把内存序从 Acquire/Release 加强到 SeqCst 没有解决问题：故障点只从 441 移到 443，随后又在 444 重现。QEMU monitor 显示底层锁状态已经空闲，但仍有 4 个等待者，因此根因不是“写入不可见”，而是用于判断条件的函数本身有副作用并造成丢唤醒。

2.2.7 LoongArch 架构特有根因

LoongArch 最初只在 RISC-V 路径上完整验证。多核重负载暴露出页表 MAT、NR/NX 属性编码不完整，以及 TLB refill 对空 PTE 的处理错误；这会把普通按需缺页变成无法恢复的 refill 循环。LoongArch 的 1 ms 定时器若每次都轮询 smoltcp 和扫描所有 interval timer，也会产生高频全局锁竞争。其 VirtIO-Net 又使用 PCI 设备而非 RISC-V 的 MMIO 设备，需要正确枚举 BAR 并复用同一网络状态机。

2.2.8 编译成功后仍可能丢失全部分数

首次官网样本曾完成编译，却没有通过最终产物运行。嵌套 QEMU 和 shell 会对 stdin/stdout、Unix socket 与网络 socket 执行 `F_SETFL(O_NONBLOCK)`、`sendmsg/recvmsg`、`sendto/recvfrom`。旧 File trait 对不支持状态修改的描述符返回失败，使子 QEMU 启动前退出；socket 路径又直接解引用用户指针，在 COW 或跨页情况下可能返回错误。

LoongArch glibc 还根据 `AT_HWCAP` 判断非对齐访问能力。没有报告 `HWCAP_LOONGARCH_UAL` 时，最终静态产物在启动阶段走入不兼容路径。因此必须把“编译返回 0、产物存在、产物大小、目标架构和产物实际启动”作为一条不可拆分的验收链。

3 修复或优化的设计与实现

3.1 核心修复与优化设计

3.1.1 时间、计时与热中断路径

内核新增独立单调时间源，`CLOCK_MONOTONIC` 和 `/proc/uptime` 只使用自启动以来的单调计数；`CLOCK_REALTIME` 使用单独的 epoch offset。实时时钟校准不再改变 sleep、futex timeout 或 BuildStorm 计时。

LoongArch 保留 1 ms 定时精度，但网络轮询只在 10 ms 调度 tick 或设备需要时进行；smoltcp 时间戳来自同一单调源。interval timer 维护全局最近截止时间的原子 hint，只有当前时间达到该 hint 才扫描任务并发布新的最早截止时间，避免每次中断遍历全部进程。

3.1.2 进程、线程、fd 与等待语义

任务结构显式维护稳定的 TGID、共享地址空间对象和共享 `files_struct`。实现 `CLONE_FILES` 后，fd 表的引用计数、`FD_CLOEXEC`、`close_range` 和 `execve` 关闭协议以共享对象为边界；`execve` 前先收敛同线程组其他线程，再替换地址空间。pipe 改为 64 KiB 环形缓冲，并先在内核 scratch 中完成用户复制，避免持设备锁触发缺页。

`wait4/waitid` 按线程组扫描和回收子进程，子进程退出时唤醒整个父线程组中可能等待的线程。signal 发送遵守 `SchedulerLock->TaskLock` 锁序，在提升 Ready 任务优先级前重新核对状态；若目标已在另一个 CPU 上 Running，只发送重调度 IPI，不再建立第二个队列所有权。重复入队断言被保留，用于捕获新的调度器错误。

3.1.3 futex、epoll 与 IPC

私有 futex key 使用 `address_space_id+page/offset`，只在明确的 `MAP_SHARED` 映射上使用共享后备对象标识。wait、wake、requeue、超时和信号中断都从所有旧 key 清理节点；线程退出按 Linux 语义写零 `clear_child_tid` 并 futex wake。

RISC-V64 与 LoongArch64 的 `epoll_event` 按 Linux UAPI 使用 16 字节布局, events 位于偏移 0, data 位于偏移 8。EPOLLET 不再只比较两次瞬时电平, 而为 file/socket 维护 `readiness_generation`, 事件在两次采样之间出现又消失时仍不会丢边沿。内核同时补齐 `eventfd`、AF_UNIX `socketpair`、`sendmsg/recvmsg` 以及必要的 poll 行为，为 cargo jobserver、rustc 和嵌套 QEMU 提供通用 IPC。

3.1.4 虚拟内存、按需装入与 COW

用户访问改为 “在 VM 协议下准备页面，再复制，发现映射代际改变则重新准备” 的统一路径。内核写用户页时可在持有地址空间写同步的情况下触发 COW; `fork`、`mprotect` 和 `munmap` 不能插入校验与复制之间。socket、pipe、wait、stat 和 exec 参数都复用该安全复制接口，而不是直接解引用用户指针。

`MADV_DONTNEED` 依次移除 PTE、收集活动 CPU、执行远端 TLB shootdown，再释放物理帧; `munmap/mprotect/mremap` 使用相同活动地址空间协议。resident frame map 分片，减少多核缺页和解除映射时对单一 BTreeMap 的争用。

ELF `PT_LOAD` 段和匿名用户栈改为按需分配。第一次缺页按 VMA 权限、文件偏移、非页对齐头尾和 BSS 零填充规则加载; 只读/可执行文件页进入全局文件页缓存，私有可写页保持进程私有并沿现有 COW 处理。启动阶段也不再提前清零全部 8 GiB 物理内存，只在帧真正分配给用户前完成零化。该设计与 Linux demand paging 一致，不减少程序可见数据。

3.1.5 VFS 语义与缓存一致性

同一文件的 size、代际和脏页状态以 superblock 与 inode 号为 key 共享，解决多打开实例、稀疏写和截断后的可见性。inode 元数据通过单次 raw snapshot 取得，避免 `stat` 为 mode、uid、gid、size 和时间戳重复遍历路径。正 dentry、负 dentry、inode snapshot 和可复用句柄模板都有代际; create、unlink、rename、truncate 或 writeback 后按父目录 inode 和目标 inode 精确失效，不能把陈旧负缓存当作 ENOENT。

create/remove/rename 以父目录 inode 为原子命名空间锁域，并在 lwext4 事务内再次核对源、目标和 expected inode，消除 TOCTOU。rename 目录时不再按新建目录重复增加链接计数; `ftruncate` 更新 inode size 而非文件 offset; 打开后

unlink 的对象由 inode 引用继续存活；目录缩短后迭代器把空项正确视作 EOF，避免在 `readdir` 中断言或越界。

3.1.6 文件页缓存、块缓存与 I/O 合并

文件读取增加按 inode 分片的页缓存和固定 64 KiB 顺序预读。相同页的并发 miss 通过 in-flight 状态合并，只由一个线程读取，其他线程等待代际完成；truncate、写入和回收会使旧代际失效。写路径使用按 inode 共享的延迟写回页，读可以直接命中尚未落盘的最新脏数据。

块设备层把线性槽位查找改为预分配哈希索引，并使用全局 CLOCK 淘汰近似 LRU。脏块达到高水位时批量写到低水位，连续物理块最多合并为 128 KiB 的逻辑写回，再按驱动 64 KiB bounce 边界提交。高低水位滞回避免每个 miss 都触发一轮写回；写回前后都检查块代际，保持 journal 和并发读写一致性。

3.1.7 allocator、调度器与锁

物理帧 allocator 和动态内核 heap 按平台 CPU 数分片。分配优先当前 CPU 的 shard，耗尽时再尝试其他 shard；释放根据指针或物理地址回到原 shard，而不是按当前 CPU 回收，因此任务迁移和跨核析构不会破坏 buddy 边界。8 B 到 4 KiB 的小对象增加每 CPU 规格化空闲链表，缓存溢出仍按地址归还原始 shard。固定 64 MiB 启动堆作为早期启动和兜底；大内存机器从物理内存追加按 CPU 切分的动态区，增长上限为 960 MiB。在 8 GiB guest 中该上限仍保证至少约 7 GiB 可用于用户页和文件页缓存。

周期性时间片到期时参考 Linux `need_resched` 思路：若本 CPU 没有本地竞争任务、亲和性仍合法且调度器状态可无阻塞读取，就继续运行当前任务；存在竞争、主动 yield、阻塞、退出、亲和性变化或锁读取失败时仍调度。睡眠读写锁的等待谓词只读取 `reader_count/writer_count`，不再调用有副作用的 `try_read/try_write`；状态发布、等待者登记和唤醒顺序保持明确的 acquire/release 关系。

3.1.8 LoongArch 页表、TLB 与设备

LoongArch PTE 按权限设置 MAT、NR 和 NX 属性，TLB refill 遇到无效或空 PTE 时进入统一缺页路径，而不是装入错误映射或自旋。共享地址空间维护

active CPU mask, 修改映射后用序列号、IPI 和 ACK 完成远端全 TLB shootdown, 空闲 hart 也能响应。PCI VirtIO-Net 完成设备枚举、BAR 映射和中断/轮询接入, 继续复用 RISC-V 的 smoltcp socket 状态机。

3.1.9 最终产物启动与确定性构建

默认 File trait 的 `set_flags` 对无法改变状态但允许忽略的描述符按 Linux 兼容行为返回成功，真正支持非阻塞的 terminal、pipe、eventfd、Unix socket 和网络 socket 保存并应用 `O_NONBLOCK`。`sendto/recvfrom/sendmsg/recvmsg` 使用安全用户复制并保持消息边界。LoongArch auxv 报告 `HWCAP_LOONGARCH_UAL`，使 glibc 与实际硬件非对齐访问能力一致。

构建系统为 RISC-V64 和 LoongArch64 分离 lwext4 C 产物和用户态输出，顶层双架构 `makeall` 使用确定顺序，避免并发 Make 复用错误架构的静态库。该增量位于 `8632a3e1`，不改变官网成功样本的内核运行语义，只降低最终打包时的交叉架构污染风险。

3.2 题 2.2 修改覆盖矩阵

本节把开发记录、git 历史和最终源码逐项对齐。表中的“覆盖”表示该能力已进入最终 `finalsubmission`；“实验记录”表示只用于诊断且已回退，不计入最终实现。

表 3.1 BuildStorm 2.2 全部修改覆盖审计

子系统	最终实现与 Linux 语义	主要源码位置	状态
评测入口	CAgent 后运行完整 2.1+2.2；双盘 chroot；保留网卡；结束后关机	`user/src/bin/initproc.rs`, `Makefile`	覆盖
计时	monotonic/realtime 分离；真实 `/proc/uptime`；不修改评分时间	`os/src/time.rs`, `os/src/fs/proc.rs`	覆盖

子系统	最终实现与 Linux 语义	主要源码位置	状态
动态装载	glibc PT_INTERP、PIE/TLS、auxv、相对路径、symlink 与解释器路径	os/src/mm/memory_set.rs, os/src/task/task.rs, os/src/syscall/process.rs	覆盖
按需 exec	ELF PT_LOAD 与用户栈 demand paging；BSS 和非页对齐边界	os/src/mm/memory_set.rs, os/src/task/task.rs	覆盖
clone/exec	CLONE_VM、CLONE_FILES、线程组与 exec 线程收敛	os/src/task/task.rs, os/src/syscall/process.rs	覆盖
fd 生命周期	共享 files_struct、FD_CLOEXEC、close_range、dup/fcntl	os/src/task/task.rs, os/src/syscall/fs.rs	覆盖
pipe	64 KiB 环形缓冲、阻塞/非阻塞、锁外用户复制	os/src/fs/pipe.rs, os/src/syscall/fs.rs	覆盖
futex	私有地址空间 key、MAP_SHARED key、requeue、timeout 清理、clear_child_tid	os/src/syscall/process.rs, os/src/task	覆盖
等待与退出	wait4/waitid 线程组子进程范围；父线程组唤醒；退出资源顺序	os/src/task/mod.rs, os/src/syscall/process.rs	覆盖
signal/调度	Ready–Running 原子复查、IPI 重调度、无竞争时跳过无效周期切换	os/src/task/manager.rs, os/src/task/processor.rs	覆盖
epoll/eventfd	16 字节 epoll_event ABI、EPOLLET generation、eventfd 与 poll	os/src/fs/epoll.rs, os/src/fs/eventfd.rs	覆盖

子系统	最终实现与 Linux 语义	主要源码位置	状态
Unix socket	socketpair、消息边界、sendmsg/recvmsg、非阻塞语义	os/src/fs/unix_socket.rs, os/src/net/syscall.rs	覆盖
网络 socket	sendto/recvfrom 安全用户复制；双架构 VirtIO-Net；smoltcp 单调时钟	os/src/net, os/src/drivers/net	覆盖
用户内存	VM 同步下 prepare/-copy/retry，消除 fork COW 与 syscall 写回竞态	os/src/mm/memory_set.rs, os/src/syscall	覆盖
COW 与共享 VM	fork COW、活动 CPU mask、远端 TLB shoot-down 与 ACK	os/src/mm/memory_set.rs, os/src/task/processor.rs	覆盖
mmap 系列	mmap/munmap/mprotect/mremap/MADV_DONTNEED 真正撤 PTE 和回收页	os/src/syscall/process.rs, os/src/mm/memory_set.rs	覆盖
文件页缓存	只读页共享、in-flight 合并、64 KiB 预读、代际失效	os/src/mm/file_page_cache.rs	覆盖
ext4 基本语义	稀疏文件、共享 inode size、打开后 unlink、truncate/stat 正确大小	os/src/fs/inode.rs, os/src/fs/vfs.rs, lwext4_rust	覆盖
目录命名空间	create/remove/rename 原子化；目录链接计数；替换和 EOF 边界	os/src/fs/vfs.rs, lwext4_rust/c/lwext4/src/ext4.c	覆盖
VFS 缓存	正/负 dentry、inode snapshot、句柄模板、父目录和 inode 代际	os/src/fs/vfs.rs, os/src/fs/inode.rs	覆盖
延迟写回	按 inode 共享脏页、最新脏读、高低水位 flush 与一致性失效	os/src/fs/inode.rs, os/src/fs/block_device_stream.rs	覆盖

子系统	最终实现与 Linux 语义	主要源码位置	状态
块缓存	哈希索引、全局 CLOCK、连续块合并、64 KiB 设备边界	os/src/fs/block_device_stream.rs	覆盖
帧 allocator	地址分片、按原 shard 回收、按需清零	os/src/mm/frame_allocator.rs	覆盖
内核 heap	每 CPU 小对象缓存、64 MiB 启动堆、最高 960 MiB 动态 shard	os/src/mm/heap_allocator.rs, os/src/config.rs	覆盖
VM 锁	无副作用 reader/writer count 等待谓词，修复 LoongArch 丢唤醒	os/src/sync/lock.rs, vendor/polyhal/src/utils/mutex_no_irq.rs	覆盖
热中断	网络 10 ms 节拍、interval timer 最近 deadline hint	os/src/main.rs, os/src/task/manager.rs	覆盖
LoongArch MMU	PTE MAT/NR/NX、空项 TLB refill、IPI TLB 刷新	vendor/polyhal/src/pagetable/loongarch64.rs, vendor/polyhal-trap	覆盖
LoongArch ABI	AT_HWCAP=HWCAP_LOONGARCH_UAL 与 glibc 启动一致	os/src/task/task.rs	覆盖
终端与产物	F_SETFL/O_NONBLOCK、默认 File set_flags、嵌套 QEMU socket 用户复制	os/src/fs/stdio.rs, os/src/fs/mod.rs, os/src/net/syscall.rs	覆盖
双架构构建	lwext4 和 user 产物按 target 隔离，顶层确定顺序打包	lwext4_rust/build.rs, lwext4_rust/c/lwext4/Makefile, user/Makefile, Makefile	覆盖
反向候选	大 bounce、自适应预读、fault-around、可写 ELF 共享、批量 allocator 等	表 4.3 与 git/开发记录	实验记录

覆盖矩阵包括从工具链启动到最终产物运行所需的全部 2.2 增量，也包括性能实验中确认无效并回退的分支。它不把更早的初赛 LTP、lmbench 或面向其他测试的修改重复包装为 BuildStorm 优化；只有直接支撑复杂构建或保证其前置回归的内容进入本文。

4 实验分析

4.1 实验口径

所有 A/B 都使用相同架构、核数、内存、磁盘镜像和工作量，相邻运行以减少宿主波动。负数代表耗时、请求或等待下降。Mac 本地的 x86_64 QEMU 经 Rosetta 运行，绝对 wall-clock 不能外推到官网原生评测机；因此本地表主要用来证明方向和排除回归，最终时间结论只采用官网 guest `/proc/uptime`。

4.2 最终保留优化的逐项 A/B

表 4.1 最终保留优化的独立实验

修改	负载/指标	修改前	修改后	变化
全局哈希 CLOCK 与合并写回	`FS_QUICK` 耗时	71.61 s	44.30 s	−38.14%
同上	块写请求	16,926	2,475	−85.38%
同上	写 DMA 时间	12.818809 s	1.757420 s	−86.29%
固定 64 KiB 顺序预读	`FS_QUICK` 耗时	56.47 s	39.07 s	−30.81%
同上	读请求数	15,115	3,465	−77.08%
inode 共享脏读缓存	512 文件完整读取	5,130 ms	946 ms	−81.56%
VFS 正负缓存和句柄模板	`FS_QUICK` 耗时	39.17 s	34.70 s	−11.41%
同上	raw inode 查询	5,915	383	−93.52%
冷目录缓存与代际	2,500 文件扫描	15,362 ms	6,095 ms	−60.32%
匿名栈按需分配	四路 rustc 并发段	389.36 s	377.78 s	−2.97%
ELF `PT_LOAD` 按需装入	`cargo --version` 中位数	9.07 s	6.36 s	−29.88%

修改	负载/指标	修改前	修改后	变化
同上	完整 2.1 cargo build	111.24 s	102.16 s	−8.16%
每 CPU 小对象缓存	四路 rustc 并发段	126.95 s	125.54 s	−1.11%
同上	buddy 锁获取	168,379	33,448	−80.14%
动态 heap 分片	`FS_QUICK` 耗时	35.45 s	32.39 s	−8.63%
同上	1 GiB 并行退出	30.98 s	8.18 s	−73.60%
同上	buddy 累计等待	145.893205 s	0.161926 s	−99.89%
按需周期调度	调度器锁调用	235,213	169,668	−27.87%

块缓存优化后，平均写请求从约 8,977 字节增至 62,052 字节，约为 6.91 倍；脏块淘汰次数从 14,095 降为 0。inode-directed rename 使局部持锁时间下降 42.73%，卷锁等待下降 18.62%，但整轮 wall-clock 基本不变，因此只把它作为锁粒度收益，不夸大整体加速。

4.3 并发故障的阶段性证据

故障/候选	修改前表现	修复后的证据	结论
信号优先唤醒竞态	RV 在 `0/446` 重复入队 panic	重新核对 Ready/Running 后越过原点至 `41/446`，后续完整通过	确定修复
256 MiB heap 耗尽	`41/446` 出现 lwext4 malloc failed 与 OOM	扩容、分片与释放路径修复后完成全部 446 单元	确定修复
VM 锁 Acquire/Release	LA 从 `438--441/446` 移到 `443/446` 仍停滞	仅移动故障窗口	回退
VM 锁全部 SeqCst	LA 越过 443 后在 `444/446` 重现空锁等待者	排除单纯内存序假设	回退
无副作用等待谓词	空闲锁仍有等待者	使用 reader/writer count 后自然越过 `438--445/446`，预构建与正式构建完成	保留
目录迭代器收缩	LoongArch readdir panic	空目录项按 EOF 处理后 446 单元预构建完成	保留

表 4.2 从可重复故障到完整构建的证据链

4.4 反向实验与回退决策

性能工作中不以“锁次数更少”代替真实结果。以下候选均通过基本语义检查，但没有进入最终源码；记录它们是为了说明设计选择经过反证，而不是只保留有利样本。

表 4.3 未保留候选及回退原因

候选	A/B 结果	根因判断	状态
VirtIO bounce 64 KiB 扩至 128 KiB	`FS_QUICK` 34.77 s 增至 38.03 s (+9.38%)	大缓冲复制成本增加约 175%，抵消请求合并	回退

候选	A/B 结果	根因判断	状态
顺序预读自适应扩至 128 KiB	相同工作量无稳定收益	短 ELF 和离散元数据产生无效预读	回退
文件缺页 fault-around 聚簇映射	cargo 6.20 s，处于波动；页表锁调用 +63.21%	预读命中不代表进程会访问相邻页	回退
私有可写 ELF 页初始共享	cargo +5.17%；四路 rustc +1.47%	很快发生 COW，新增写缺页和页表修改	回退
目录删除查询结果复用	两轮均值 35.85 s 增至 38.55 s（+7.52%）	局部查询占比小，事务内复核仍不可省	回退
allocator 8/32 批处理	125.54 s 增至 126.35 s（+0.65%）	buddy 合并延长关中断临界区	回退
allocator 2/8 批处理	125.54 s 增至 127.19 s（+1.31%）	锁次数下降但累计等待上升	回退
文件缓存 ghost admission	双 rustc 83.09 s 增至 88.39 s	一次性数据多于短期重复访问	回退

4.5 完整长测与官网结果

本地干净覆盖层长测中，RISC-V64 的 446 单元预构建从 137 分 47 秒降至 120 分 24 秒，约缩短 12.6%；其正式 release 构建耗时 3459.62 秒，产物 1,681,000 字节。LoongArch64 预构建为 54 分 51 秒，正式构建 1642.49 秒，产物 1,714,568 字节。由于本地宿主并发负载和 Rosetta 影响，这些数字用于证明全链路完成，不作为官网基线。

官网最终结果如表 1.2。RISC-V64 日志在 36 分 59 秒完成 release profile，最终 guest 计时为 2270.92 秒；LoongArch64 在 27 分 40 秒完成，最终计时 1700.89 秒。两边都打印对应目标三元组的产物路径、非零字节数和 `run=OK`，随后测试组结束。

```
BUILDSTORM_RESULT mode=multi status=OK rc=0 cores=8 \
  elapsed_s=2270.92 artifact=target/riscv64gc-unknown-linux-musl/
  release/arceos-helloworld bytes=1681000 run=OK
```

```
BUILDSTORM_RESULT mode=multi status=OK rc=0 cores=12 \
  elapsed_s=1700.89 artifact=target/loongarch64-unknown-linux-musl/
  release/arceos-helloworld bytes=1714568 run=OK
```

5 AI 使用说明与可复现步骤

5.1 AI 辅助使用说明

本项目使用 OpenAI Codex 系列大模型和 AI Agent 作为开发辅助工具。AI 的实际参与并不限于文字润色：它协助检索 Linux/UAPI 语义，阅读仓库和日志，生成过部分 Rust/C 修改草案、诊断插桩、压力测试和文档初稿，并在长测期间帮助归纳锁、页表、系统调用与文件系统之间的关联。与 AI 的交流记录、每次命令输出、git 提交、开发文档和官网日志均被保留，可用于追溯哪些建议进入过候选代码。

核心目标、总体架构取舍和验收标准由我们掌握并决定。团队明确要求修复必须符合 Linux 通用语义，不能按 crate 名称或评测输出绕过工作；队员决定测试优先级、分支发布时机、双架构和回归边界，审查 AI 给出的补丁，并通过本地 QEMU、Docker、官方镜像和官网评测决定保留或回退。最终代码的锁序、地址空间生命周期、页缓存代际、ext4 事务和双架构页表逻辑均可由队员解释和现场修改，AI 不替代团队承担设计、验证和提交责任。

AI 生成内容曾出现错误或不完整判断。例如，单纯加强 VM 锁内存序只移动了卡死位置；扩大 VirtIO bounce、allocator 批处理和可写 ELF 共享在微基准中反而变慢；早期也曾把宿主资源终止与 guest 故障混在同一轮长测里。团队通过 QEMU monitor、锁计数、紧邻 A/B、新鲜磁盘覆盖层和官网日志发现这些问题，没有通过删除断言、伪造时间或修改评分脚本掩盖失败，而是回退无收益候选并继续定位到无副作用等待谓词、COW 同步、目录 EOF 和产物启动 ABI 等根因。

本文在撰写时同样让 AI 协助整理并总结了完整对话记录、`documents` 下的开发日志和 git 历史，但所有数字均来自已有测试输出或官网结果，不由模型估算。我们复核了最终分支、提交边界、日志标记和评分公式；官网成功结果是独立于 AI 的最终验证来源。

5.2 可复现开发过程与最终验收

本节的目标不是只复现“最终内核能够通过一次评测”，而是按相同输入重演“基线失败或变慢—诊断取证—单项修复或优化—同条件 A/B—完整验收”的开发过程。最终耗时会受宿主机和 QEMU 版本影响，但问题位置、诊断计数、候选方案的保留/回退结论和双架构功能结果均应保持一致。

5.2.1 复现材料、版本和不变量

复现本过程需要完整 git 历史、官方 RISC-V64 与 LoongArch64 BuildStorm 数据盘、官网输出 `Riscv -11.txt` 与 `LoongArch -11.txt`、容器镜像标识及其 digest。仅提供最终源码和一张已经写入过的磁盘，只能验证最后结果，不能重演开发过程。仓库中的 documents/BuildStorm 题二复杂构建适配与验证记录.md 按时间记录每轮假设、证据、短测和回退；documents/BuildStorm 官网输出 10 问题定位与终端非阻塞修复.md 记录最后产物启动问题。

每一轮运行前记录并固定以下不变量：提交 ID、容器/QEMU 版本、架构、CPU 数、内存大小、官方数据盘 SHA-256、根盘 SHA-256、是否开启诊断 feature，以及 guest 串口完整输出。开发期的本地双架构比较使用相同官方数据盘、`-smp8-m8G` 和同一根盘生成规则；官网最终结果保留平台实际核数，即 RISC-V64 为 8 核、LoongArch64 为 12 核。每次运行使用数据盘的新副本或写时覆盖层，禁止复用已经包含 Cargo 增量缓存、临时文件或失败残留的磁盘。

5.2.2 从基线重演提交演进

先检查提交顺序和每次改动的边界；下面的命令把报告中列出的关键提交逐一定位，而不是仅检查当前 `HEAD`：

```
git fetch origin --prune
for commit in df511f76 ecc1a5fe 6ec1e609 b469dbd1 90803c5e \
  01c7091b dae87457 d293494c 881565f2 09642b80 18d0c340 \
  266bd1aa 8632a3e1; do
  git show --no-patch --format=fuller "$commit"
done
git diff --check df511f76..266bd1aa
git diff --stat df511f76..266bd1aa
```

在独立 worktree 中对每一个阶段执行 `gitswitch--detach<commit>`，再运

行下一小节的相同短测和双盘命令。df511f76 是 BuildStorm/Q2.1 的八核基础；266bd1aa 是官网成功样本的最后关键修复；8632a3e1 只处理双架构本地打包隔离。对于任一根因，应同时阅读对应开发记录中的“诊断—根因—修复—验证”小节，并用 gitshow<commit>--<source-path> 将记录和最终代码逐项对齐。

5.2.3 建立干净输入并采集定位证据

首先确认环境与数据盘，并为每轮创建新副本。下面以支持 reflink 的 Linux 文件系统为例；不支持时用等价的完整复制或 qcow2 写时覆盖层：

```
git rev-parse HEAD
docker image inspect zhouzhouyi/os-contest:20260510
qemu-system-riscv64 --version
qemu-system-loongarch64 --version
sha256sum /path/to/buildstorm-riscv64.img \
          /path/to/buildstorm-loongarch64.img
mkdir -p tmp/repro
cp --reflink=auto /path/to/buildstorm-riscv64.img tmp/repro/rv-round.img
cp --reflink=auto /path/to/buildstorm-loongarch64.img tmp/repro/la-round.img
```

定位阶段先打开诊断 feature，而不是直接反复运行数小时长测。该 feature 只输出锁竞争、futex、页表和块 I/O 的诊断信息，不改变 Linux ABI；其结果用于定位，不用于报告最终耗时：

```
make run-dual ARCH=riscv64 QEMU_SMP=8 QEMU_MEM=8G \
  SMP_DIAGNOSTICS=1 TEST_DISK=tmp/repro/rv-round.img ROOT_DISK=disk.img \
  2>&1 | tee tmp/repro/rv-diagnostic.log
make run-dual ARCH=loongarch64 QEMU_SMP=8 QEMU_MEM=8G \
  SMP_DIAGNOSTICS=1 TEST_DISK=tmp/repro/la-round.img ROOT_DISK=disk-la.img \
  2>&1 | tee tmp/repro/la-diagnostic.log
```

将串口中的错误系统调用、任务状态、锁等待、块请求数和 QEMU monitor 快照与第 1 章及开发记录的对应小节比对。只有在新鲜磁盘上稳定出现的 guest 证据才作为根因；Docker/Rosetta 的资源终止、手工中止或旧覆盖层损坏均应单独标注，不能作为内核缺陷结论。

5.2.4 按同条件 A/B 重演优化决策

对一个候选修改，先在前一提交或关闭该候选的 worktree 建立基线，再在候选 worktree 运行相同短测。两次运行必须使用同一架构、核数、内存、官方数据

盘副本和 rootfs 生成方式；每一次只能改变一个候选。每轮保存提交 ID、命令行、guest `/proc/uptime`、串口日志、块请求/字节数和锁等待统计，并按表 4.1 计算加速比 $= T_{before}/T_{after}$。

短测覆盖文件创建与 rename、truncate/稀疏文件、共享 inode、COW、`MADV_DONTNEED`、线程组 wait、futex requeue、epoll ET、同核公平性和 8 核 SMP 压力；随后才运行工具链和完整构建。若候选不能同时保持 Linux 语义和同条件性能收益，就按表 4.3 回退。这样可重演“为什么保留或放弃某项优化”，而不只是得到一个最终二进制。

5.2.5 重建最终产物并执行官方双盘验收

关闭诊断 feature 后，在最终提交上重新生成两个架构的内核和根盘，避免诊断代码或旧产物影响正式结果：

```
git switch finalsubmission
git status --short
make all
file kernel-rv kernel-la
ls -lh kernel-rv kernel-la disk.img disk-la.img
```

把官方 BuildStorm 镜像作为 `TEST_DISK`，根盘使用刚生成的对应架构镜像；RISC-V64 和 LoongArch64 的最小复现命令分别为：

```
make run-dual ARCH=riscv64 QEMU_SMP=8 QEMU_MEM=8G \
  SMP_DIAGNOSTICS=0 TEST_DISK=/path/to/buildstorm-riscv64.img \
  ROOT_DISK=disk.img
make run-dual ARCH=loongarch64 QEMU_SMP=8 QEMU_MEM=8G \
  SMP_DIAGNOSTICS=0 TEST_DISK=/path/to/buildstorm-loongarch64.img \
  ROOT_DISK=disk-la.img
```

若评测平台传入不同核数，以平台 QEMU 参数为准，judge 会读取最终 `cores` 字段。最终验收不能只看 cargo 最后一行，必须同时出现以下四类证据：

```
TOOLCHAIN_RESULT status=OK
MINIBUILD_RESULT status=OK
BUILDSTORM_RESULT ... status=OK ... bytes=<at least 512000> run=OK
#### OS COMP TEST GROUP END buildstorm-glibc ####
```

最终官网计时应仅以两份官网输出为准，不能把不同宿主机的 wall-clock 数字混入表 1.2。完成上述步骤后，可从基线和诊断日志重演问题定位与开发决策，

并从最终提交、干净磁盘和官方标记得到最终结果。

5.3 总结与后续方向

ScintillaOS 最终解决的不是一个单点 syscall，而是一条跨越动态链接、线程组、futex、用户内存、ext4、VirtIO、allocator、调度和 LoongArch MMU 的复杂构建链。性能上的最大收益来自文件系统大粒度路径重构：哈希 CLOCK 块缓存、合并写回、固定顺序预读、inode 共享脏页和 VFS 代际缓存；稳定性上的关键则是 COW 用户复制、线程组生命周期、无副作用 VM 锁等待谓词、目录 EOF 和最终产物 ABI。

官网双架构工具链、minibuild、完整构建和产物启动全部通过，证明这些修改形成了可运行的通用 Linux 兼容能力，而不是只让评分脚本输出成功。后续仍可在不改变工作量和计时语义的前提下继续降低 RISC-V64 的 ext4 全卷串行区、改进块层异步队列和实现更成熟的页回写调度；但任何新优化都应继续遵守本文的原则：先证明 Linux 语义，再做同条件 A/B，出现退化立即回退，最后以双架构完整产物运行作为发布门槛。